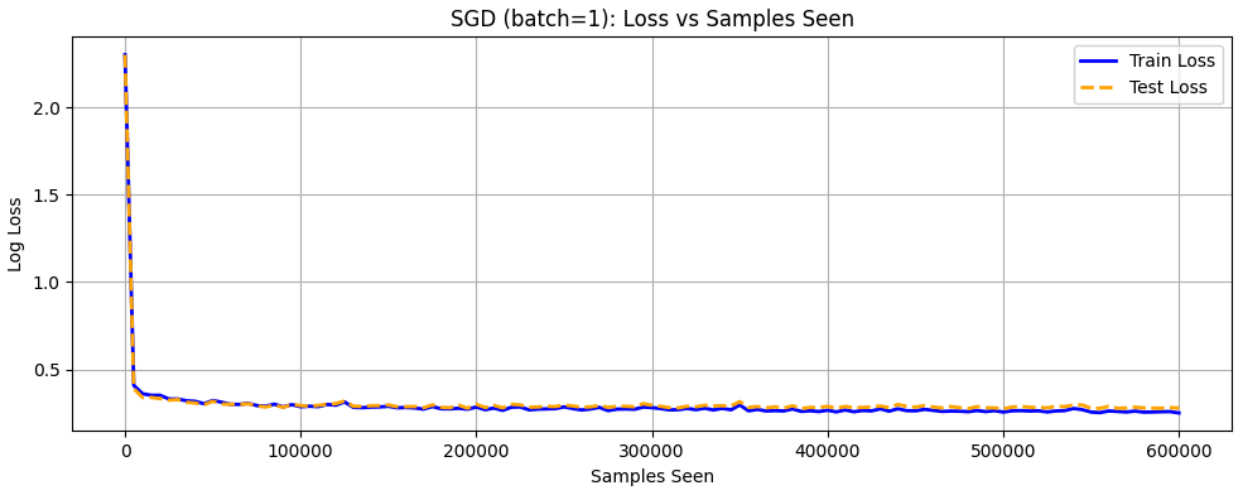
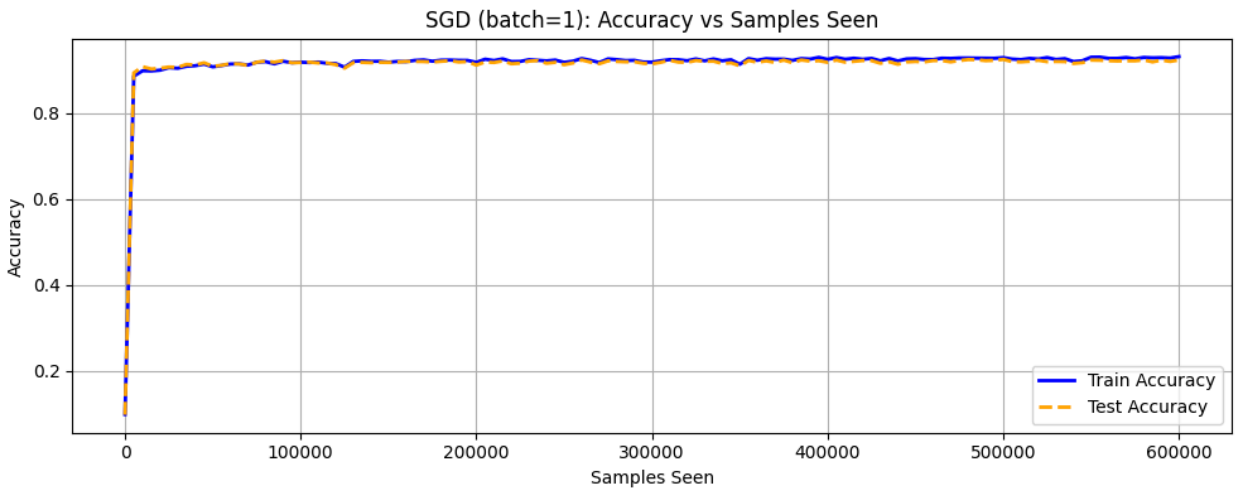
Aishu Sivamurugan
EE541 HW 6

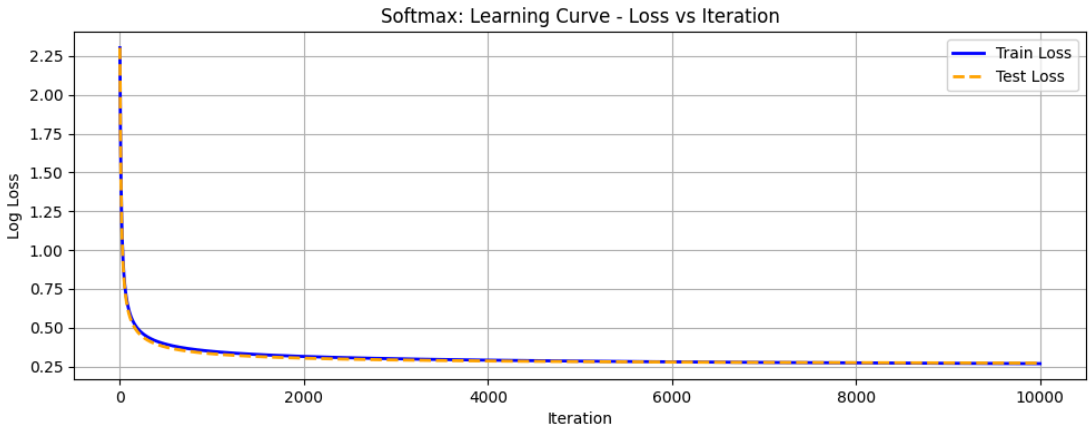
1. **Implement SGD with mini-batch size of 1 (i.e. compute the gradient and update weights after each sample). Record the log-loss and accuracy of the training set and test set every 5,000 samples. Plot the sampled log-loss and accuracy values on the same (respective) figures against the batch number. Your plots should start at iteration 0 (i.e. include initial log-loss and accuracy). Your curves should show performance comparable to batch gradient descent. How many iterations did it take to achieve comparable performance with batch gradient descent? How does this number depend on the learning rate? (or learning rate decay schedule if you have a non-constant learning rate).**

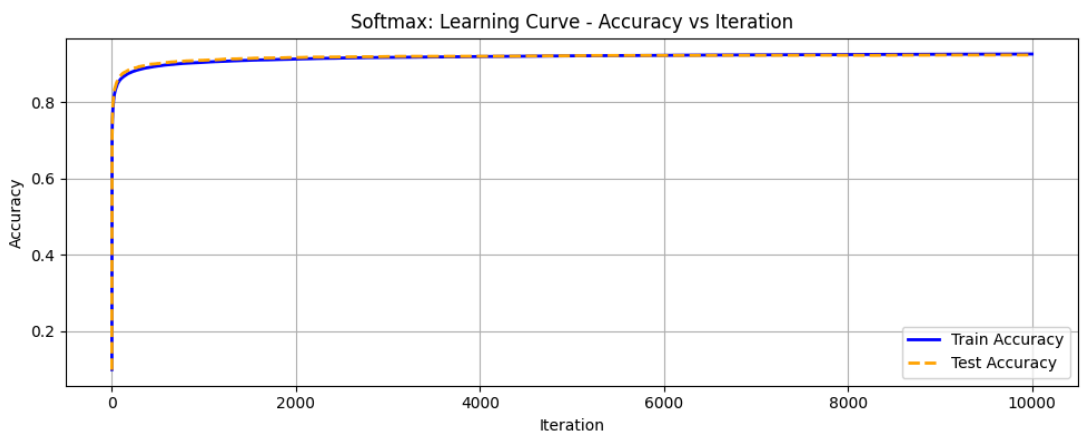
- SGD with batch size 1 achieved comparable performance to batch gradient descent (~92% test accuracy) after approximately 50,000 samples, corresponding to roughly 1 full epoch through the training data. By comparison, batch gradient descent reached comparable accuracy after approximately 500 iterations, but each iteration processes all 60,000 samples , making SGD far more efficient in terms of total computation.

- With a constant learning rate of 0.1, the curves show characteristic noisiness throughout training, visible in both the loss and accuracy plots. A smaller learning rate would likely reduce noise and improve the final performance. A decay schedule where we would gradually decrease the learning rate over time, taking larger steps early and more precise steps later for a smoother, quicker, convergence.


SGD (batch=1): Loss vs Samples Seen
Train Loss
Test Loss
Log Loss
2.0
1.5
1.0
0.5
0
100000
200000
300000
400000
500000
600000
Samples Seen


SGD (batch=1): Accuracy vs Samples Seen
Train Accuracy
Test Accuracy
Accuracy
0.8
0.6
0.4
0.2
0
100000
200000
300000
400000
500000
600000
Samples Seen



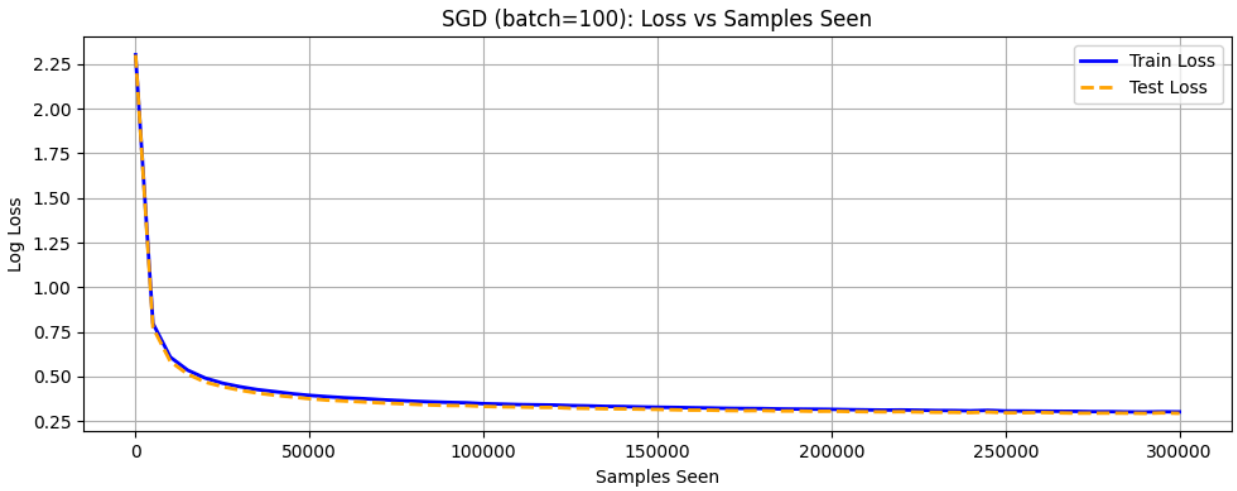


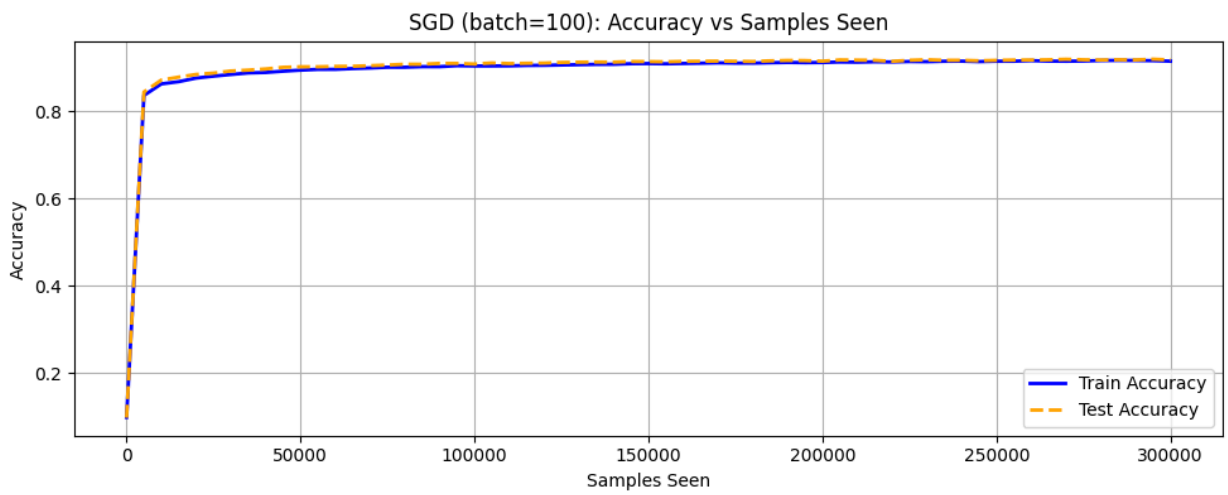
2. **Compare (to batch gradient descent) the total computational complexity to reach a comparable accuracy on your training set. Note that each iteration of batch gradient descent costs an extra factor of operations where is the number of data points.**

- Batch Gradient Descent reached 92% accuracy. It took about 500 iterations × 60,000 samples per iteration = 30,000,000 sample evaluations
- SGD batch reached 92% accuracy. It took about 50,000 updates × 1 sample per update = 50,000 sample evaluations
- 30,000,000/50,000 = 600
- SGD batch=1 is about 600x more computationally efficient than batch Gradient Descent to reach the same accuracy.

3. **Implement SGD with mini-batch size of 100 (i.e. compute the gradient and update weights with accumulated average after every 100 samples). Record the log-loss and accuracies as above (every 5,000 samples – not 5,000 batches) and create similar plots. Your curves should show performance comparable to batch gradient descent. How many iterations did it take to achieve comparable performance with batch gradient descent? How does this number depend on the learning rate? (or learning rate decay schedule if you have a non-constant learning rate).**

- SGD with mini-batch size of 100 achieved similar performance to batch gradient descent (~92% test accuracy) after approximately 100,000 samples, corresponding to roughly 2 full epochs through the training data. This is slightly slower than batch size 1 in terms of samples seen, but the convergence is smoother and computational time is much faster. This is because averaging the gradient over 100 samples produces a much more reliable estimate of the true gradient direction before each weight update, leading to speedier convergence.
- With a constant learning rate of lr=0.1, the model reached a final train accuracy of 91.47% and test accuracy of 91.76% after 5 epochs (300,000 samples. Batch size 100 handled lr=0.1 without instability, larger batches produce less noisy gradients so larger learning rates are tolerable. A decay schedule could still potentially improve final performance by allowing finer adjustments near convergence, but was not necessary here to achieve comparable results to batch Gradient Descent.





-

4. **Compare the computational complexity to reach comparable performance between the 100 sample mini-batch algorithm, the single-point mini-batch, and batch gradient descent.**

	Samples to ~92% acc	Cost per update	Total sample evaluations
Batch GD	~500 iterations	O(60,000 × 784 × 10)	~30,000,000
SGD batch=1	~50,000 updates	O(1 × 784 × 10)	~50,000
SGD batch=100	~1,000 updates	O(100 × 784 × 10)	~100,000

Key takeaways:

- Batch GD vs SGD: Batch GD is the most computationally expensive. It took forever to run as it must process all 60,000 samples before taking a single step.
- SGD batch=1 vs batch=100: batch=1 reaches comparable accuracy using fewer total sample evaluations (~50,000 vs ~100,000). However in practice batch=100 is faster. It took over 90 minutes for batch=1, less than five minutes to fully process batch=100.
- Overall the best was mini-batch SGD with batch=100. It reached similar accuracy as the others, and processed in a fraction of the time, making it the best when considering all of these tradeoffs.